Development of a 3D CNN for Automated Recognition and Classification of Volleyball Movements

Sarah Jordan, G22S

16.10.2026

1 Abstract

Artificial Intelligence has experienced a considerable upturn in usage in professional sports, thus facilitating analysis and performance improvement. Volleyball, too, has started profiting from this development and is implementing more and more AI in daily practice as well as competition. It has increased efficiency in comparison to formerly practised manual input of data for scouting.

This project focuses only on the first part of scouting: the recognition and classification of volleyball actions. A pipeline was developed to recognize and classify three different volleyball actions: reception, set and spike. A custom dataset of 300 clips was created from three matches with six recordings. All 300 clips were run through an R3D-18 network pretrained on the Kinetics-400 dataset in order to extract features. A small classifier head consisting of a dropout layer and a linear layer was added and trained to classify the three actions. The R3D-18 was frozen, so only the weights and biases of the head were updated, using CrossEntropyLoss and the Adam optimizer. The model reached an accuracy of 0.919 ± 0.034 on the held-out test set. Since this split contained clips from the same recordings on both sides of the net, the result was suspected to be too optimistic. Leave-one-recording-out cross-validation was therefore performed to see how the model would generalise to unseen recordings. The accuracy dropped to 0.854 ± 0.090, which shows that domain shift affects the model. The random split has therefore considerably overestimated the performance on this small, self-created dataset. It is necessary to split the data by recording in order to evaluate what the model actually learned.

Contents

1 Abstract

2 Introduction

2.1 Background . . .
2.2 Motivation . . .

3 Theoretical Background **1**

3.1 The Game of Volleyball . . . 1
3.1.1 Tactics . . . 1
3.1.2 Techniques . . . 2
3.2 Scouting . . . 3
3.2.1 Conventional Scouting vs. Modern Scouting . . . 3
3.3 Machine Learning and Deep Learning . . . 5
3.3.1 Use of ML and DL in Volleyball in the Market . . . 5
3.4 Neural Networks . . . 6
3.4.1 Forward Propagation and Loss Function . . . 6
3.4.2 Back Propagation . . . 7
3.4.3 3D Convolutional Neural Network . . . 7
3.4.4 Residual Network . . . 8
3.5 Preprocessing . . . 9
3.6 Evaluation of the Model . . . 9
3.6.1 Key Metrics . . . 9

4 Methods and Technologies **11**

4.1 Classes . . . 11
4.1.1 Source of Data . . . 11
4.1.2 Criteria of Choosing . . . 13
4.1.3 Cutting and labelling . . . 15
4.2 Preprocessing . . . 15
4.2.1 Data Preparation . . . 15
4.2.2 Pilot Run and Visual Inspection . . . 16
4.2.3 Output and Storage . . . 17
4.3 Dataset and Split . . . 17
4.3.1 Grouped Stratified Split . . . 17
4.3.2 Class Distribution . . . 17
4.4 Training and Validation . . . 18
4.4.1 Transfer Learning . . . 19
4.4.2 Backbone, Feature Extraction and Augmentation . . . 20
4.4.3 Head Classifier . . . 20
4.4.4 Loss Function, Back Propagation, Optimizer . . . 21

4.4.5 Dropout, Early Stopping . . . 21
4.4.6 Choice of Hyperparameters . . . 21
4.4.7 Training and Validation Curves . . . 22
4.4.8 Evaluation Protocol . . . 22

5 Results **24**
5.1 Training Performance . . . 24
5.2 Choice of Weight Decay . . . 24
5.3 Performance on Test Set . . . 25
5.4 Per-Class Performance . . . 26

6 Discussion with Outlook, Reflection **27**
6.1 Robustness of the Data . . . 27
6.1.1 Trimming and Sampling . . . 27
6.1.2 Data Source Distribution . . . 27
6.2 Training Performance, Overfitting . . . 28
6.3 Augmentation . . . 28
6.4 Testing results . . . 29
6.4.1 Accuracy, Confusion Matrix, F1 . . . 29
6.4.2 ROC-AUC, Precision, Recall . . . 29
6.5 Generalisation to Unseen Recording . . . 29
6.5.1 Problem to Idea . . . 29
6.5.2 Method . . . 30
6.5.3 Results . . . 30
6.5.4 Discussion . . . 31
6.6 Reflection . . . 31
6.6.1 Process . . . 31
6.6.2 Custom Model vs. Pretrained Model . . . 32
6.6.3 Reflecting on the Usage of AI . . . 32
6.6.4 Troubles and Issues . . . 32
6.6.5 Technical Skills Learned . . . 33
6.6.6 Time Management, Workload . . . 34

7 Conclusion **35**

Glossary **I**

List of Abbreviations **IV**

Sources **V**

Hilfsmittelverzeichnis **X**

Materials **XIII**

2 Introduction

2.1 Background

Video analysis in volleyball has become an indispensable tool for improved performance in terms of tactical and technical skills. Until recently the analysis has been performed by live watching or rewatching video material to hand annotate and evaluate every single action of a e.g. game, which was laborious and time consuming.[1] But in recent years new AI driven technologies have emerged and have since then facilitated or even erased the need for manual analysis.

Machine and deep learning-driven pipelines with Convolutional Neural Networks have shown to be time efficient and accurate for action recognition and therefore been found suitable as a tool for improving the analysis of volleyball games and practices.[2]

2.2 Motivation

As a professional volleyball player and tech aficionado myself, I was amazed when this kind of technology first emerged and was later used in our games and practices. I wanted to get greater insight on what such technologies are based on and if I would be able to replicate a similar, simpler tool myself, which led to the inspiration for this project.

3 Theoretical Background

3.1 The Game of Volleyball

Volleyball is a non-contact team sport which consists of up to 14 players and five staff members on the sheet per team. Each side of the court, which is separated by a net, is defined by base lines on nine times nine meters and a three meter line, which separates the back row players from the front row players. To win a match a team needs to win three sets. The team wins a set, which scores 25 points first, with a minimum lead of two points. A set can therefore theoretically be played indefinitely. In the case of both teams winning two sets each (a tie of 2-2), a fifth deciding set is played to 15 points, also with a minimum lead of two points. Each team has the right to use up to three touches to return the ball over the net. [3]

Interested readers are referred to the following website of the Swiss Volley Federation, which contain the Rule Book of the FIVB and Swiss Volley's additions:

https://www.volleyball.ch/de/wissen/volleyballregeln

3.1.1 Tactics

In elite volleyball a player is specialised in a certain role. The assignment of certain roles helps a team to maximise their chances of scoring. Furthermore, assigning a role to a player automatically leads to a better task and collective behaviour organization among the team.

Each position of the players on the volleyball court is defined by numbers from one to six and the players are forced to rotate clock-wise after winning a side-out. As shown in Figure 1.

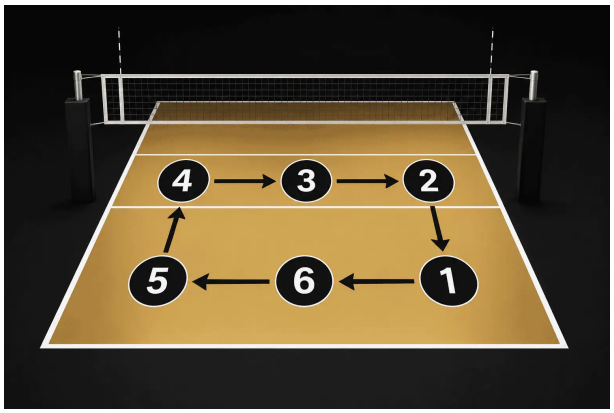


Figure 1: Volleyball basic positions

All players must position themselves in the right order of their position number they are at. After the start of the approach of the service of the opponent, the team players are free to run to their assigned position and role. Shown in figure 2 with P6 (setter on position 6). [3]

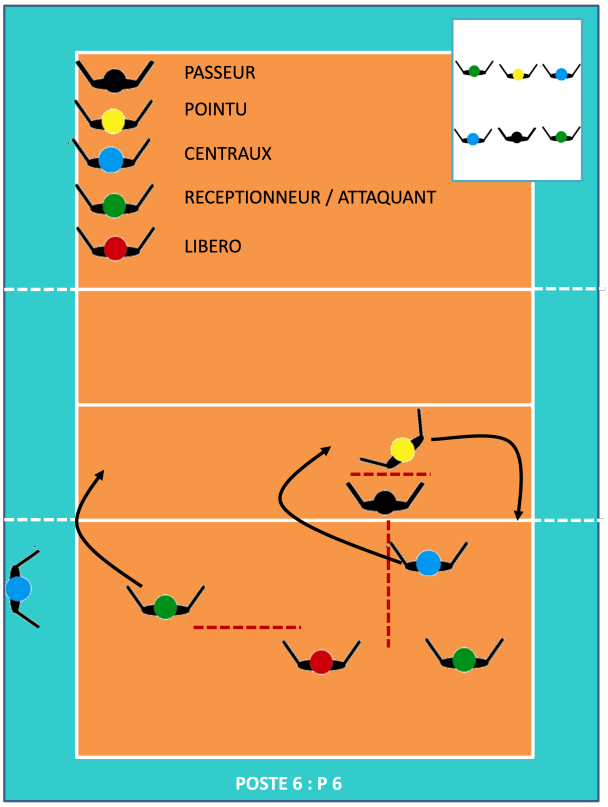


Figure 2: Setter on Position 6

In this paper we differ between 5 roles: Setter, Middle Blocker, Outside, Opposite and Libero

3.1.2 Techniques

To become a great volleyball player, a player must be well versed in all vital techniques regardless of their role in the team. We will differ between five techniques.
The rally starts with a service. Serve is therefore part of the offense. The serve is the first chance in the rally to control the speed and rhythm of the opponents game. A successful serve breaks the defense of the opponent. Two types of serves exist, one is called float and the other spin.[4]
The serve is followed by the passing of the other team. Passing is the first part of the defense. It is also divided into two subcategories, such as reception and defense. Reception is the passing act after a serve and defense is the passing act after an attack of the opposing team. It is the first touch of the three (blocking is exempt). You can pass the ball either by an overhead pass (set) or a forearm pass (bump). [4]
Reception is followed by setting, which is the second touch. Setting is a crucial part of switching from defense to offense because it determines how well the next action can be executed. The set determines who is going to attack and at what speed. The more varied and smart a player plays, the harder it is for the opposing team to play a good defense. [4]
After the setting comes the attack. Either on the net (front row players) or on the three meter line (back row players). Attacking is the third touch and is the primary way of scoring points. Successful

attacks put pressure on the opponent by controlling the speed and rhythm of the rally [5]. An attack is executed by spiking, tipping or playing a roll shot. [4]
Last comes blocking. Blocking is the primary line of defense and plays a key role in stopping or slowing down an opponent's attack. Successful blocks additionally put psychological pressure on the attack players that may cause them to make mistakes. [6]

3.2 Scouting

Scouting is the systematic process of observing, recording, and analysing every touch, action, and tactical tendency of the ball during a match [7]. It can be split into two parts. First the movement is recognized. Second its quality is assessed. By collecting data from individual actions and noting how e.g setters distribute sets in specific rotations, where attackers tend to attack and on which position or side receivers struggle to receive[8], a tactical advantage may be obtained.

Scouting is used for game preparation to find out the opponent's strengths and weaknesses. Through this information, a tactical game plan for the offense and defense can be prepared and studied. [8]
Not only is the data collected from scouting used against opponents but it is also essential to improve the own team and the individual player. The data gives coaches an objective and accurate view of the team's and player's needs for further improvement. [7]

3.2.1 Conventional Scouting vs. Modern Scouting

Conventional performance analysis relies on subjective evaluation by coaches, hand-recorded statistics, and video review after the fact, which makes it labor-intensive and limits its ability to accurately capture complex behaviors in training or competition in real time. Additionally, because feedback is delayed, analytical results cannot be quickly turned into targeted training adjustments or evidence-based decisions. [1] Figure 3

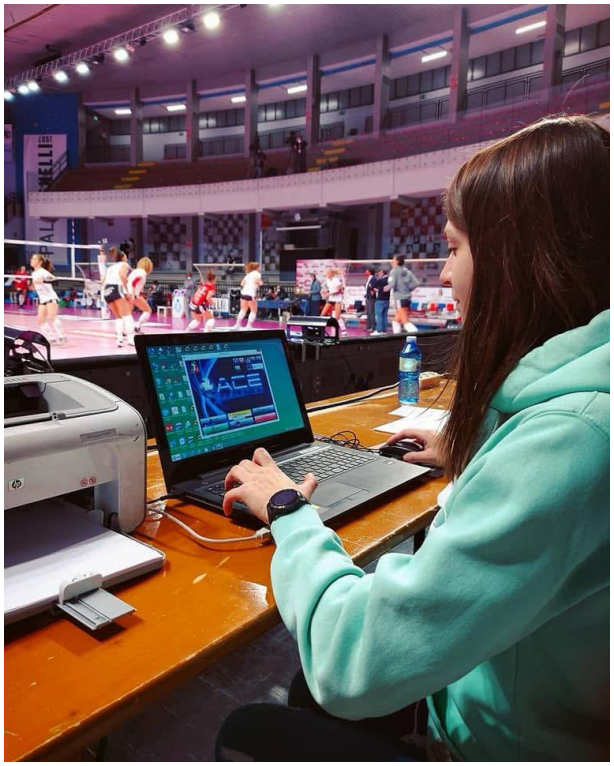


Figure 3: Manual scouting live during a game

The rise of artificial intelligence (AI) in recent years has opened up new possibilities to overcome these challenges[1]. Moreover, machine learning (ML) and deep learning (DL) are making scouting more accessible by automating tasks that once required manual effort.

3.3 Machine Learning and Deep Learning

ML and DL are both subcategories of AI. DL is a subcategory of ML. Visualized in Figure 4.

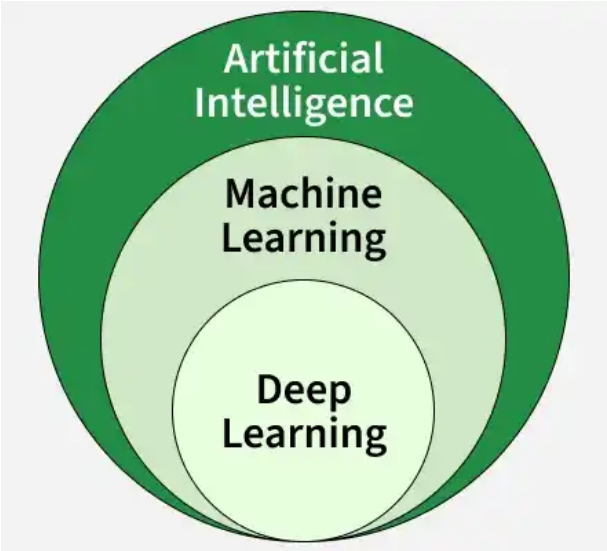


Figure 4: Overview of relation between AI, ML and DL

ML models and algorithms are tools that can learn patterns and solve exercises, by training, without the need for direct coded guidance, such as classical programming [9]. However, a ML model still relies heavily on the input of manual feature engineered data to effectively learn, unlike DL [10].

DL is more advanced than ML due to its special structure of many layers of neural networks. DL models can be given the raw data and learn complex patterns, without the need for additional manual input. [11, 9]

3.3.1 Use of ML and DL in Volleyball in the Market

Established commercial scouting tools such as VolleyStation (VS) and VolleyMetrics (VM) (powered by Hudl) had previously relied on human analysts to tag each action. Then, machine learning was applied downstream to the structured data, after feature engineering. The scouting files created by VM can then be uploaded to e.g. OpenVolleyScout, where the play can be analysed. Conversely, Balltime (powered by Hudl) is one of the tools that uses deep learning models and relies on computer-vision to recognize and filter actions of single players in games and practices. Nevertheless, Balltime does not export scouting files, which are needed to export analysis, which limited its use for match preparation at the time. [12, 13]

In February 2025, Balltime was acquired by Hudl, who had already acquired VM in 2017. By acquiring Balltime, Hudl was now able to merge the two technologies and therefore combine automated labeling with scouting files for analysis. [12]

3.4 Neural Networks

A neural network is a construct of mathematical equations that are used to learn complex patterns. Hence, the reference to the neurobiological term does not mean that the neural network (NN) works like the brain.
A typical NN consists of one input, hidden and output layer, which are all interconnected with each other. A deep neural network (DNN) has multiple hidden layers as shown in Figure 5.[9, 14]

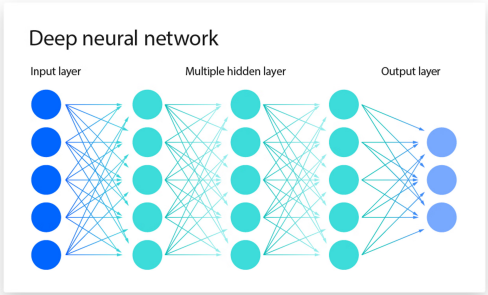


Figure 5: Structure of a NN

3.4.1 Forward Propagation and Loss Function

Forward propagation is the process of passing information through the NN, directing from the input layer through hidden layer to the output layer. The input layer receives the preprocessed data and passes it on. In the hidden layers, the data passes through linear transformation and an activation function. The output layer is the final layer and produces the model's prediction. [14, 15] A point in figure 5 represents a neuron that receives inputs. The lines in figure 5 represent connections that link neurons and carry information between them. Numeric parameters, weights (w) and biases (b), regulate the strength and influence of a connection. The higher a weight, the stronger the connection between two neurons, and therefore the more importance is given to an input. A bias is added to the weighted sum of inputs inside a neuron. It ensures that a neuron can be activated even when all input features are zero, and it shifts the input of the activation function, which gives the network more flexibility. A single neuron computes following mathematical equation:

$$z = W \times X + b \tag{1}$$

z is the result of the input vector X muliplied by the weight W with a bias b added.[16, 17]

Z is then passed through an activation function, e.g. $ReLU$, where it is decided whether the information is passed on $z > 0$ or not $z < 0$. In case of $ReLU$, the NN is made efficient, by only firing positive neurons:

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases} \tag{2}$$

$$f(x) = \max(0, x) \tag{3}$$

[18]. Without non-linear activation functions a NN would just be a linear transformation model and thus not be able to handle complex patterns. [19]

Finally, the raw NN output, the shape of vector with an arbitrary numerical value, is converted into a normalized probability distribution. All values must sum up to 1. These values are then compared against their true value, using a loss function, which depends on what a model's intent is. The loss function is like a guide to the NN model by showing the model's incorrectness through a numeric value. The smaller the value, the better the model's performance. The loss function is applied only during training, because it is a mathematical tool to improve the model's performance. [20, 15]

3.4.2 Back Propagation

To improve accuracy and performance of a model during training, back propagation is applied. Back propagation is the process where an algorithm moves backwards through the model to adjust weights and biases for optimal values, using a loss function and an optimizer, during training. Adam is favoured for complex models with large datasets, because it adapts the effective step size of how far a weight actually moves for each parameter automatically. This makes training less sensitive to the choice of the initial learning rate. [21, 22, 23]

3.4.3 3D Convolutional Neural Network

A Convolutional Neural Network (CNN) is a type of DL algorithm that uses several layers to extract features from data such as images, audio and video, and to classify it. Their architecture is split into three types of main layers: A convolutional layer, pooling layer and fully-connected (FC) layer. [24]

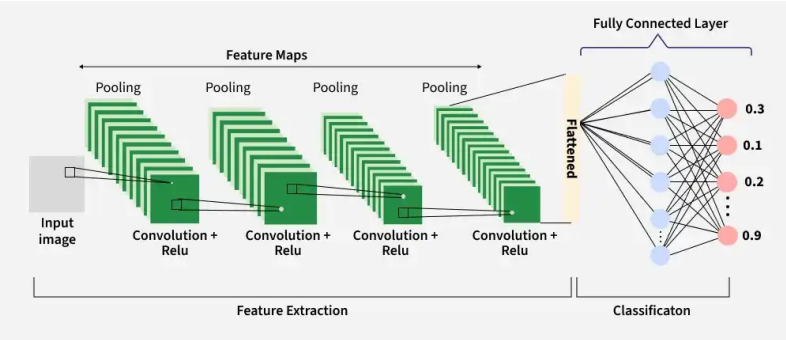


Figure 6: Structure of a CNN

A 3D CNN is needed for spatio-temporal inputs like videos. The inputs are either in the form of a video or feature map (in deeper layers) [25]. The input size is 4-dimensional $C \times D \times H \times W$, where C represents the number of RGB channels, D the depth (number of frames) of the clip, H and W the height and width of the frame [26].
The convolutional layer is the core building block of a CNN and accounts for most of the network's computation. A 3D convolutional layer extracts spatial and temporal features from its input. To do

so, a learnable filter (kernel) of size $3 \times 3 \times 3$ slides over the video clip along its width, height, and time dimensions. As shown in Figure 7

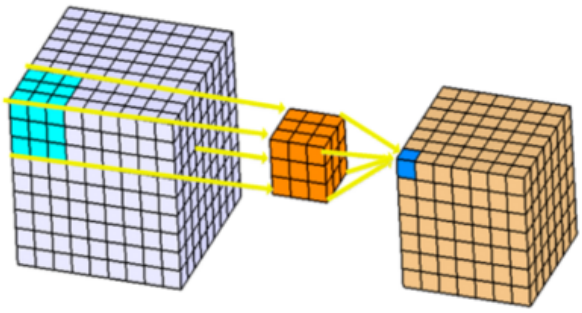


Figure 7: Kernel sliding over 3D data

At each position, the filter computes a weighted sum of the values it covers. The resulting values form a feature map, which highlights where a specific pattern appears in the clip. By learning the filter weights during training, the layer can detect increasingly meaningful spatio-temporal patterns. [27]

Pooling layers reduce the size of feature maps, by only retaining important features in the map (features with the maximum activation value) in each region. Making the network computationally more efficient. [28]

The FC layer is connected to each neuron of the layer before. It combines and processes features from the layer prior to recognize patterns. Finally, it maps the learned features to one score per class, which a softmax function converts into class probabilities. [29]

3.4.4 Residual Network

Residual network (ResNet) is a type of CNN model that enables the construction of very deep neural networks, without performance degradation [30]. By employing residual blocks with skip connection to bypass layers, it allows the gradient, in back propagation, to flow directly, enabling extremely deep networks with over 152 layers, compared to standard CNNs. [31]

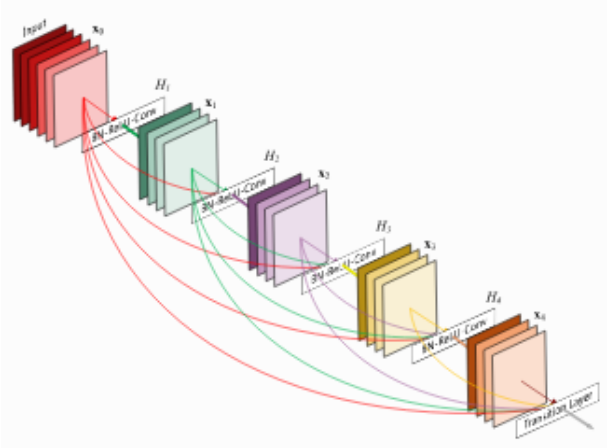


Figure 8: Structure of a ResNet model

3.5 Preprocessing

Before data is fed to a NN, it needs to undergo preprocessing for the individual use case. Preprocessing can include steps like converting, trimming, standardization, object detection metadata analysis, augmentation and transformation into tensors. It is a crucial step and the basis for good training of NNs. [32]

3.6 Evaluation of the Model

A major issue in DL is the lack of transparency of the model. Inputs and predictions can be observed, but not the reasoning that connects them. Therefore evaluation metrics are used to ensure the model provides accurate prediction for its respective use case. [33]

3.6.1 Key Metrics

The performance of a classification model can be assessed with several metrics that are all derived from the counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Accuracy is the proportion of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{4}$$

Precision measures how many of the samples predicted as positive are actually positive, and is therefore high when the model produces few false positives:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{5}$$

Recall measures how many of the actual positive samples the model identifies correctly, and is high when there are few false negatives:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{6}$$

The F1 score is the harmonic mean of precision and recall and combines both into a single value, which is useful when a balance between false positives and false negatives is needed:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{7}$$

The ROC curve plots the true positive rate against the false positive rate as the classification threshold is varied over its entire range. It therefore describes how well a model separates two classes independently of any single threshold. The AUC stands for the measured area under the ROC curve. [34] Finally, the confusion matrix compares the actual classes with the predicted ones. Each cell shows how many samples of a given true class were assigned to a given predicted class, which makes both correct predictions and specific misclassification visible at a glance. [35]

4 Methods and Technologies

4.1 Classes

The 3D CNN was given three different volleyball techniques/actions to recognize and classify. One is a forearm pass during reception, made by any player. In this paper referred to as 'reception'. One class is overhead pass, made by the setter. In this paper referred to as 'set'. Lastly, the third class was a spike during attack, made by any player. In this paper referred to as 'spike'. Those three techniques were chosen because they represent a 'typical order' of a side-out.

4.1.1 Source of Data

The goal was to provide my own dataset, since I could select from which teams, games, players and actions the data would be provided, hence having control of the quality and visibility of the actions. Thus ensuring a clearer classification for the model.

The tools VM (with a premier account) and VS (with a Pro account) were used to download .mp4 videos and .dvw files of a whole game, reupload and merge them, filter and choose actions, and download the filtered action. The tools VM and VS were chosen, because usage was already familiar and the needed subscription was available. Moreover, both are being used worldwide by elite volleyball teams.

Table 1 shows which games were selected.

Table 1: List of games with respective recording

Date of game (dd/mm/yy)	Team 1; Recording Number	Team 2; with Recording Number
23/05/26	Italy National Team (Women's);1072	Turkey National Team (Women's);1077
03/05/26	VakifBank ISTANBUL;3812	Eczacibasi Dynavit ISTANBUL; 3815
01/06/26	Canada National Team (Women's);1087	Japan National Team; 1084

VM was used to search games and download their respective .mp4 and .dvw files. The second step is to upload both files to VS.
To import the .dvw file click the following order shown in figure 9 . Click on 'select file' and select the desired file.

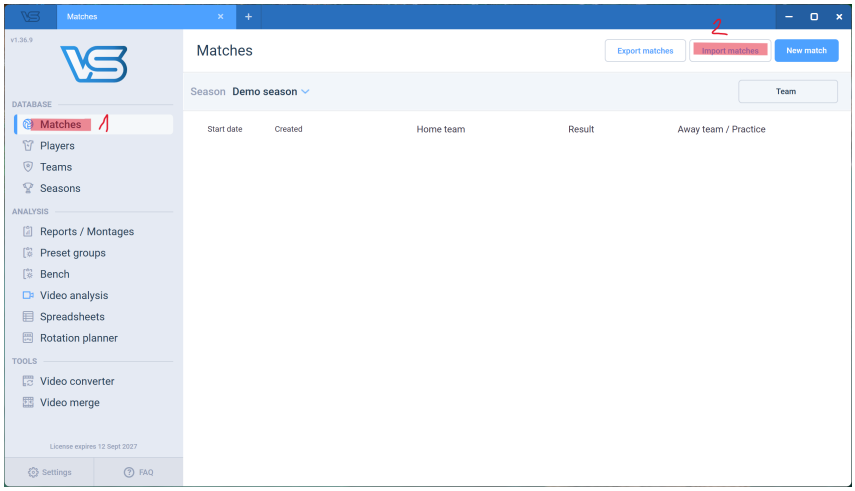


Figure 9: First Step

Next step is to merge the .dvw file with the .mp4 file. By clicking on the yellow marked symbol in figure 10 , the corresponding video can be selected.

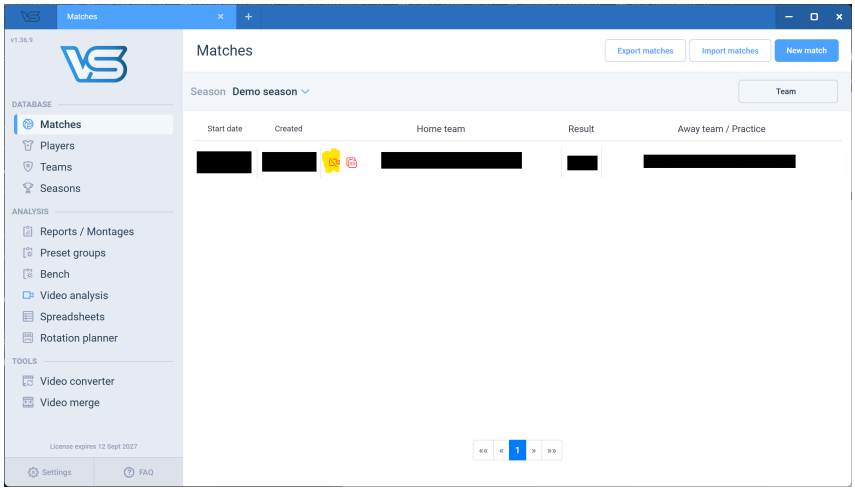


Figure 10: Second step

Then click on ’Video analysis’ on the left hand side. To select a match for analysis click on the top bar on the right of ’close’. To choose the intended game click first on ’Season’ to select the Season the game is stored in, then on ’Team’ and lastly on ’Game’. Shown in Figure 11.

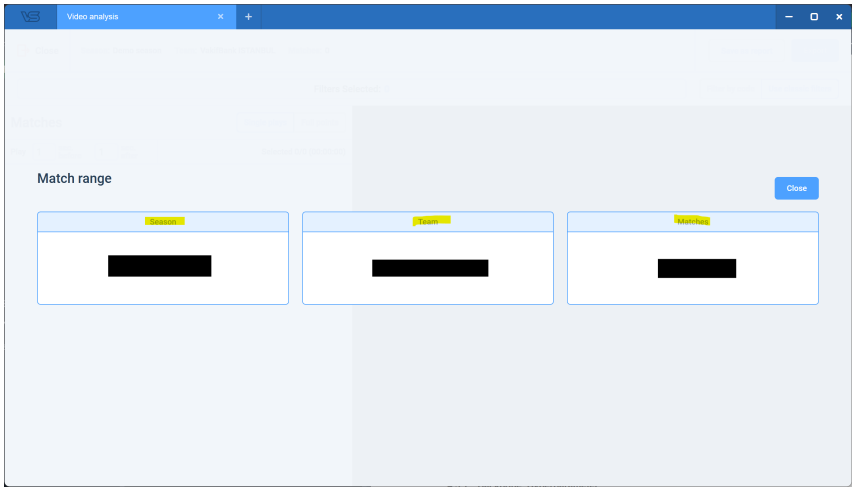


Figure 11: Third step

It is recommended to use 'classical filters' to filter actions. By clicking on 'Filter Selected' the filters can be selected. Every clip of this specific action will be listed in the left bar. Yellow marked boxes allow you to select how many seconds before and after the action should be shown. By clicking on the left side in the listed action, the clip can be exported in form of a .mp4 file. Demonstrated in figure 12.

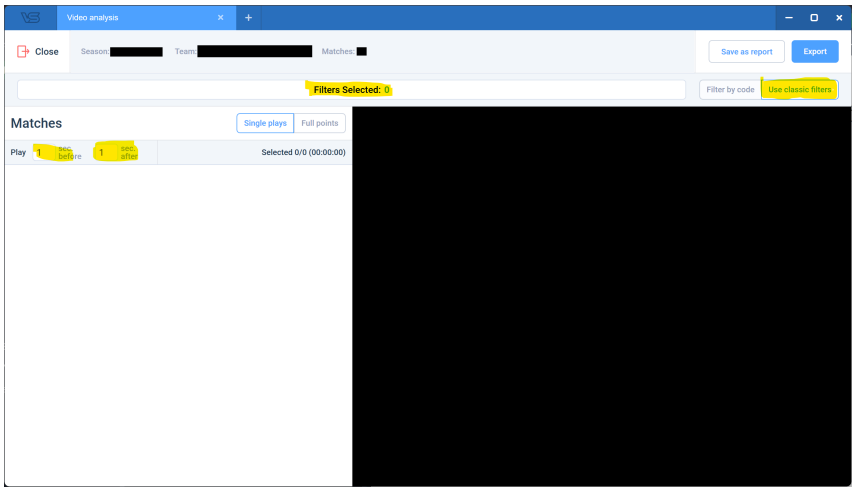


Figure 12: Fourth step

4.1.2 Criteria of Choosing

A specific filter combination was used to ensure the action could be seen and not blocked by other objects or players.

The following filters were used, shown in figure 13, for the first class (forearm pass in reception). 'Part of Match' was selected differently for each team as the team had to face backwards towards the camera.

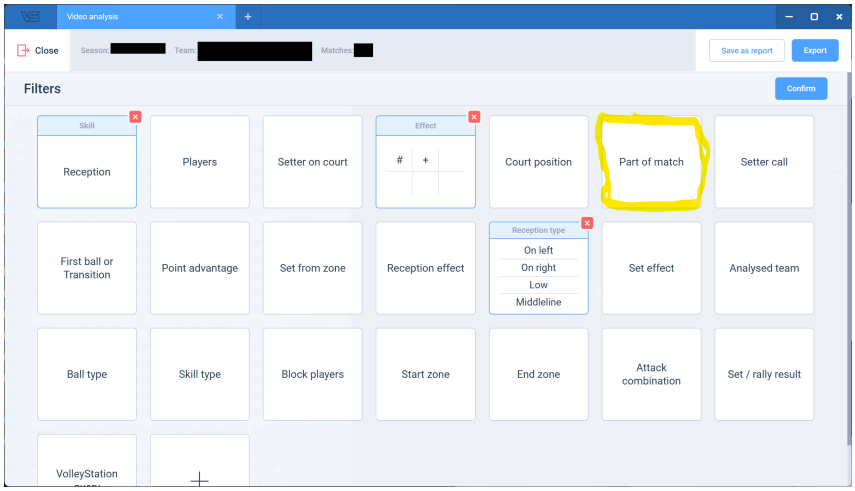


Figure 13: Choice of filter for the first class

For the overhead pass, made by the setter, class, the following filters were used (figure 14). In 'players' 'setter' can be selected, so only actions made by the setter would be shown.

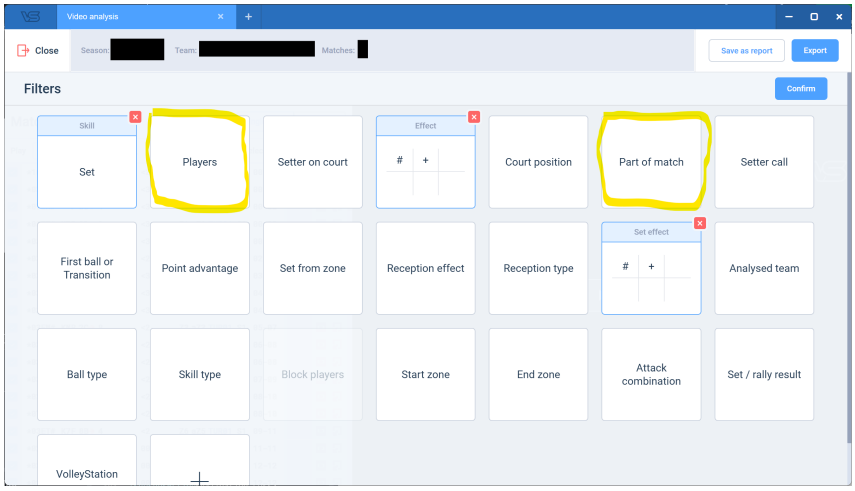


Figure 14: Choice of filter for the second class

In figure 15 is the filter selection presented for the spike class.

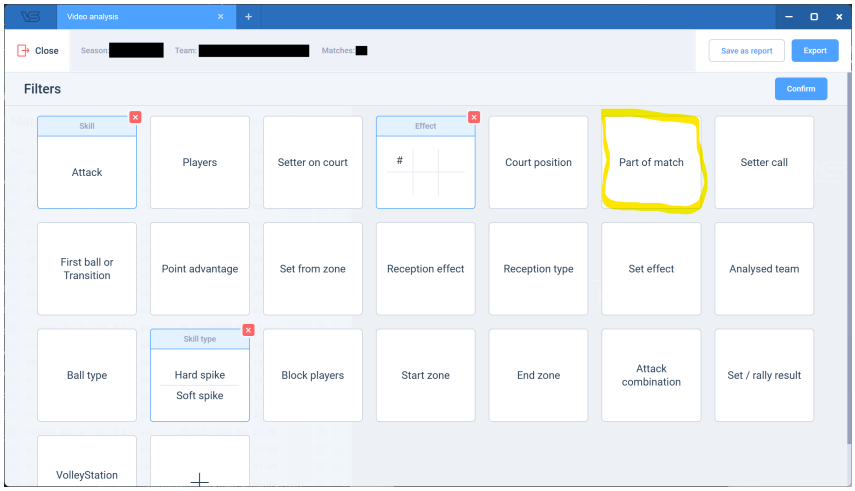


Figure 15: Choice of filter for the third class

4.1.3 Cutting and labelling

The intention was to precisely cut the clips of the action in VS and export them, as direct visual control was given to ensure that the action was given. This was not possible due to VS having an export minimum length of one second before and after the action, which was too long for showing a single action in volleyball. The precise cutting therefore had to be done in the preprocessing step instead of in VS.

Each downloaded clip was stored in its respective folder (reception, set, spike) under data/raw/¡action¿

4.2 Preprocessing

4.2.1 Data Preparation

The data set consisted of 300 clips taken from three matches. Each clip had a length of about two seconds and a resolution of 1280×720 pixels. The source material had two different frame rates: 247 clips had been recorded at 30 fps and 53 clips at 50 fps.
Before the clips could be used as input for the model, they had to be preprocessed. Each frame of a clip was decoded and converted from BGR to RGB, because OpenCV returns frames in BGR order while the pretrained model expects RGB. The clips were decoded completely and sequentially instead of seeking to a specific position, since seeking in compressed mp4 files can return the wrong frame; at a length of two seconds, decoding the whole clip was using little data storage.
In a second step, the clip was reduced to the part in which the action happens. This was necessary because the action labelling was not always precise and because the three actions did not have the same duration. A trim window was therefore defined individually for each class in a `.json` configuration file: 1.00–1.17 s for reception, 0.80–1.65 s for set and 1.25–1.35 s for spike, measured from the beginning of the clip. The windows were defined in seconds and converted into frame indices using the frame rate of each individual clip, so that the same time span was covered regardless of whether a clip was recorded at 30 or 50 fps.

From the trim window 16 frames were sampled at uniform intervals. The fixed number was required due to the pretrained weights of the r3d-18 model having been evaluated with clips of 16 sampled frames [36].
However, the clips of reception and spike at both frame rates fell back to the 'original' clip length of two seconds due to the implementation falling back by the code (figure 16) depicted form `trim_and_convert`.

```
52      start_frame = ma (variable) start_frame: int    1))
53      end_frame = max(start_frame + 1, min(end_frame, n_total))
```

Figure 16: Code for resetting trim if 16 frames are not guaranteed

4.2.2 Pilot Run and Visual Inspection

First, an initial trim window was assumed for each class. The preprocessing was then applied to two clips per class only, six clips in total. For each of these clips, four frames were taken at uniform intervals and placed next to each other in a PNG file, which also made it possible to verify the colour order and the shape of the array. If the intended action was not visible in the selected window, the configuration file was modified and the process was repeated. Only after this check was the preprocessing applied to all clips.
In order to prevent the whole data set being trimmed incorrectly, a pilot was run with a visual inspection carried out. The visual inspection was conducted by running the command: `python preview_clip.py <path>.npy`, which generated a four frame .png panorama slide. Each action visualised in figure 17 (reception), 18 (set) and 19.

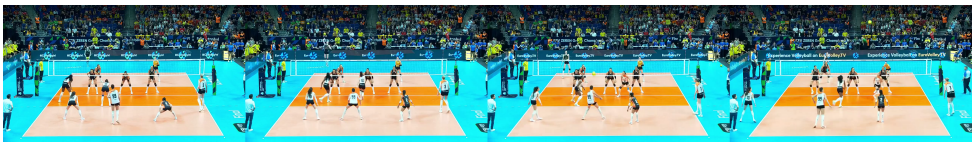


Figure 17: Reception



Figure 18: Set

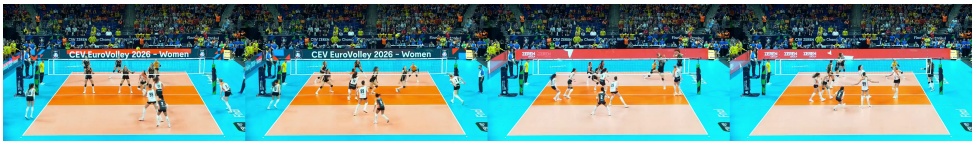


Figure 19: Spike

The final trim was an approximation of each action. Therefore, it had to be assumed that some clips show different parts of the rally than intended.

4.2.3 Output and Storage

Each preprocessed clip is stored as a `.npy` file with the dimensions $16 \times 720 \times 1280 \times 3$ and the data type `uint8`. On top, one row per clip was written to a manifest file with its respective file path, class label, group identifier and the property.

Corrupted files were collected, skipped and listed at the end of the run. Ensuring a defective video not interrupting the processing of the whole data set.

The video clips were encoded into `.npy` files, instead of another `.mp4`, which makes the data lossless and avoids repeated decoding during training. The cost of this decision is storage: the 303 MB of source video become 13 GB of uncompressed clips, 44.2 MB per clip. The new files were stored in data/processed/¡action¿. `manifest.csv` was the main reference listed in colums with rows of `path, label, group_id, n_frames, height, width and source fps`. The splitting is based on the `group_id` of the clip.

4.3 Dataset and Split

4.3.1 Grouped Stratified Split

The data set was divided into a training, a validation and a test set in a ratio of $70\,\%/15\,\%/15\,\%$. The split was stratified, which means that the proportion of the three classes is identical in all three subsets, and it was carried out per group rather than per row, so that no clip can appear in more than one subset. For each class, the group identifiers were shuffled with `random.Random(42)` and then divided according to the three fractions; the fixed seed makes the split reproducible. Splitting by `group_id` prevents data leakage. [37] If several clips originated from the same rally, nearly identical footage could otherwise appear in the training and in the test set, and the resulting metrics would be too optimistic. In the current data set, `group_id` corresponds to the file name, so each clip forms its own group; the mechanism is nevertheless part of the pipeline and would take effect if several clips were extracted from one rally.
Clips from the same match are still distributed across all three subsets. The model therefore sees the same venue, the same camera perspective and the same teams during training as during testing.

4.3.2 Class Distribution

The table 2 shows the distribution of the three classes across the three subsets. The classes are exactly balanced, with 70 clips per class in the training set and 15 per class in the validation and test sets.
Because of such small splits, a single clip corresponds to $1/45 = 2.2$ percentage points of accuracy. Differences below this value between individual runs cannot be interpreted.

Split	reception	set	spike	total
train	70	70	70	210
val	15	15	15	45
test	15	15	15	45

Table 2: Aufteilung des Datensatzes nach Split und Aktion.

4.4 Training and Validation

Figure 20 represents model's pipeline. Which will be explained in the following chapters.

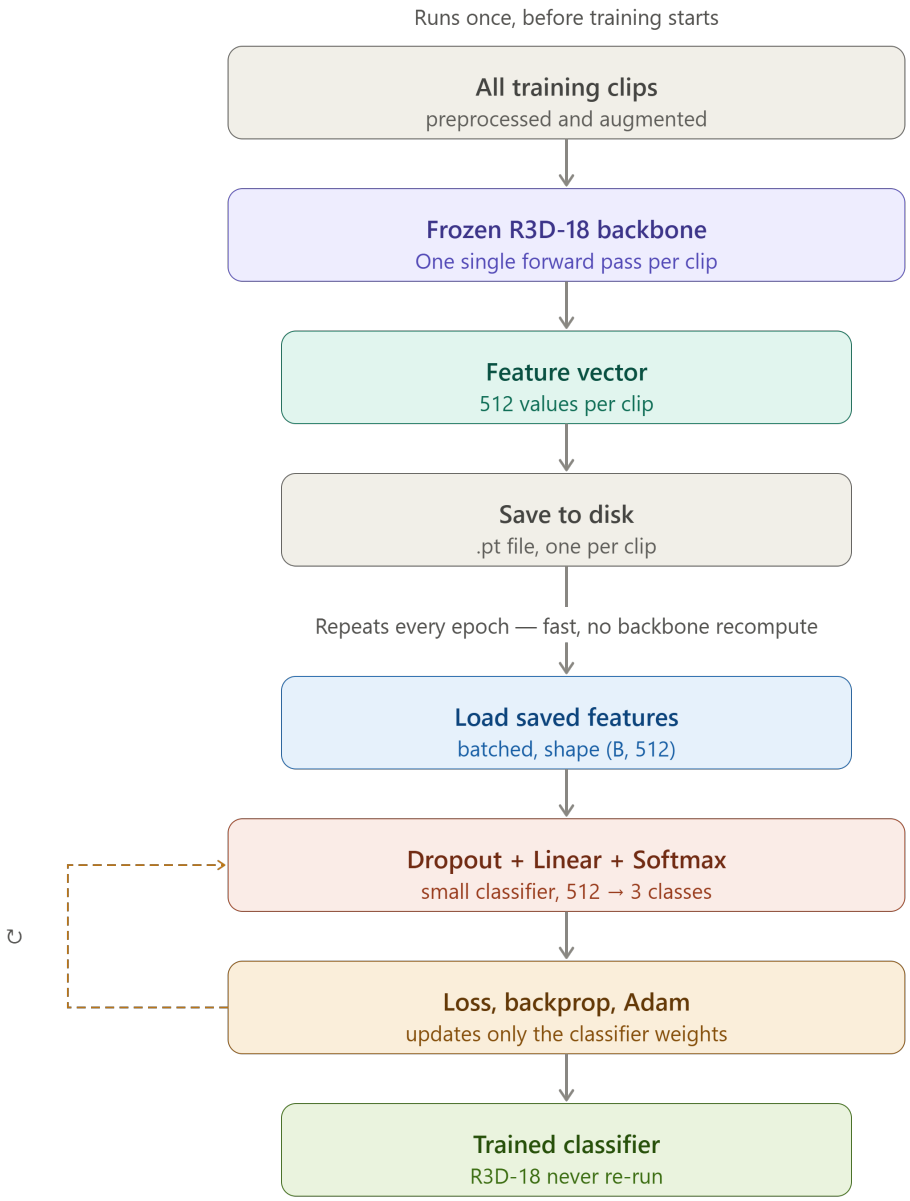


Figure 20: Pipeline

4.4.1 Transfer Learning

The initial idea was to train a 3D-CNN from scratch. This proved infeasible. The computational cost would have exceeded the scope of this project. And with a training set of 210 clips there would have been far too few for a model with 33.4 million parameters, therefore it proved also not feasible. [38] A pretrained model was used instead to transfer its learned features to a classifier. The 3D ResNet variant R3D-18 had already been trained on the Kinetics-400 data set and hence was deemed suitable. Its architecture is shown in figure 21.

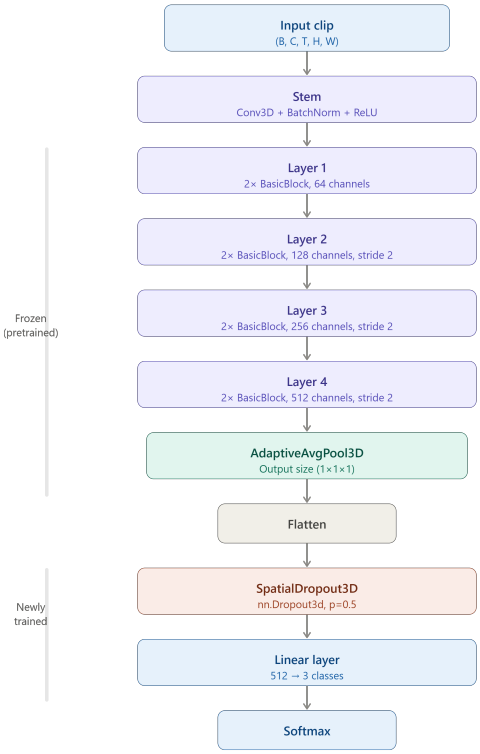


Figure 21: Architecture of the R3D-18

The Kinetics-400 data set contains approximately 300 000 videos of 400 action classes, which the model had already been trained on to learn general motion features. These features can be transferred to a new task. Such as differing between the movements of a reception, set and a spike. [36]

This dataset was chosen because it pre-trained the model on video clips and not on still images.

Hence, the R3D-18 is the frozen (refer to following chapter) backbone of the model, extracting features for a small HC.
The HC was trained on cached features. As a result, one training run took approximately four seconds instead of the considerably longer time required for training or fine-tuning the complete network, which made it possible in the first place to compare several random seeds and hyperparameter settings.



Figure 22: Code of freezing the backbone and turn into eval-mode

4.4.2 Backbone, Feature Extraction and Augmentation

`extract_features.py` was run once, passing all clips (train/val/test) through the backbone exactly once. Two steps turn R3D-18 into a feature extractor.

First, the backbone was frozen. This was done by switching off the gradient computation for all of its parameters, so that no gradients are stored for them and their weights are never updated (line 46-48 in figure 22). In addition, the model was set to evaluation mode, which keeps its internal statistics fixed as well (line 45 in figure 22). The freezing makes R3D-18 unable to learn during training and allows only the HC to be trained. Shown in figure 22
Second, the FC-layer was removed by replacing it with a layer that passes its input through unchanged: `model.fc = nn.Identity()` (line 44 in figure 22). Instead of class scores, the network now outputs the 512-dimensional feature vector of the layer before. One such vector per clip was cached as a `.npy` file on disk, together with a `<split>_features.csv` file listing them.
Video augmentation was used to increase the variety of the training data. It is only applied to the training clips, as augmenting validation or test data would distort the results. The transforms are defined in `video_dataset.py` and are imported directly by `extract_features.py`, so that the augmented clips never have to be written to disk; only their feature vectors are stored. Each random parameter (crop offset, flip decision) was chosen only once per clip and then applied to every frame of that clip, to ensure a smooth motion for the 3D convolutions. In addition to the deterministic centre crop, each training clip got four augmented copies, which results in five feature vectors per training clip.

However, the downside is that these feature vectors are cached, hence the augmentation is fixed and the model sees the same five crops and flips in every epoch.

4.4.3 Head Classifier

The architecture of the head classifier (HC) was kept simple with a `Dropout (0.5)`- and `Linear(512, 3)`-layer (line 74-77 in figure 23) to prevent overfitting. Overfitting occurs when the DL or ML model starts to memorize the train data, by learning the noise of the train data. Making it unable to generalize new data, nullifying the purpose of ML or DL. [39].

An additional classification layer (softmax) was omitted on purpose, as the loss function `CrossEntropyLoss`

internally applies log-softmax. An additional layer on the HC would double the classification and distort the gradient.

```
74      classifier = torch.nn.Sequential(
75          torch.nn.Dropout(DROPOUT_P),
76          torch.nn.Linear(FEATURE_DIM, num_classes),
77      ).to(device)
```

Figure 23: Structure of head classifier

4.4.4 Loss Function, Back Propagation, Optimizer

Cross-entropy loss was used to optimize the HC. Backpropagation computes the gradient of the loss with respect to every weight and bias by moving backwards through the network. An optimizer then uses these gradients to update the parameters. [40]

Back propagation went only through the HC, as the backbone did not need to update its weights and biases. This specific part saved the model considerable computation time.

Adam Optimizer was used because it adapts the learning rate for each parameter, so it converges reliably with the default learning rate of 1e-3 and needs little manual tuning, suiting the small HC. [41] Figure 24 showing the code.

```
79      criterion = torch.nn.CrossEntropyLoss()
80      optimizer = torch.optim.Adam(classifier.parameters(), lr=LR, weight_decay=args.weight_decay)
```

Figure 24: CrossEntropyLoss and Adam optimizer

4.4.5 Dropout, Early Stopping

Dropout and early stopping are both tools to regularize and reduce the chance of overfitting of the model during training. Dropout hinders the network on relying too much on a neuron by randomly deactivation a neuron during training, improving generalization. A dropout of 0.5 was used, meaning 50% of neurons per training step were dropped.[42]

Early stopping is a regularization technique used to monitor a model's performance by automatically stopping the training, when the validation curve stops improving, indicating the start of the model overfitting. [43] The validation loss was computed after every epoch. If it improved, the checkpoint was saved. Had the valuation loss not improved for 5 epochs (patience) then the training process would have been stopped. Val loss was used because it reacts faster compared to valuation accuracy, in my case.

4.4.6 Choice of Hyperparameters

The head classifier was trained on the frozen R3D-18 features with the settings in Table 3. Only weight decay was varied systematically; the remaining values were taken from common defaults or from the pretrained recipe and were not tuned.

Table 3: Hyperparameters of R3D-18 Backbone

Setting	Value
Backbone	R3D-18, frozen
Input	16 frames, crop 112×112
Feature dimension	512
head classifier	Dropout (0.5) $\rightarrow$ Linear ($512 \rightarrow 3$)
Loss	Cross-entropy
Optimizer	Adam [41], default learning rate 10^{-3}
Weight decay	0 (selected from $\{0, 10^{-4}, 10^{-3}\}$)
Batch size	16
Maximum epochs	50
Early stopping	Patience 5, monitored on validation loss
Learning-rate schedule	None
Seeds	0, 1, 2

Each weight decay value was used to train the HC with three different seeds. A seed is the starting value of the random number generator. It determines the weight initialization, the dropout masks and the order of the batches, so the same seed reproduces the same run. The value with the lowest mean validation loss over the three seeds was selected, and the test set was not used for this choice.

4.4.7 Training and Validation Curves

To evaluate a model's training performance training and validation curves are used. They help to detect upcoming issues before the performance of the model worsens. Loss and accuracy curves must be monitored during training and validation. They are crucial to a model's training process. The loss curve shows how the model's error evolve over several training steps. And the accuracy curve the percentage of correct predictions made over several training steps.
The data from training was retrieved and visualized by `plot_training_history` from `training_history.csv` , which was previously written by `train_classifier.py` every epoch. Making the data retrievable in case of a crash during a training run.

4.4.8 Evaluation Protocol

By running `evaluate.py` the cached test-features were loaded. The classification with softmax happens here. The seeds 0,1,2 were used. A seed is used to initialize a random number generator (RNG). Making the randomness of AI models controllable and replicable. The seed brings consistency into an AI model, making reproducibility, fair comparison and controlled creativity for developers possible. [44]

Training, evaluation and report import together the metrics from `metrics_utils.py`, making those three identical. Accuracy, precision, recall and F1 are implemented directly in PyTorch and derived

from the confusion matrix, whereas ROC-AUC and PR-AUC are taken from scikit-learn.

5 Results

5.1 Training Performance

The graphs depicted in figure 25 show the training performance of the model. All graphs show only the best epoch and stopping epoch per seed. In this case $seed = 0$ (refer to table 4). With the best epoch, indicated through a dashed line, at 14 and stopping epoch at 19 (patience = 5).

Seed	best epoch	stopping epoch
0	14	19
1	29	34
2	41	46

Table 4: Seeds, best epoch, stopping epoch.

The first graph from left shows the loss-curve of the training and validation clip, whereas the orange validation curve is below the blue training curve. The graph in the middle shows the accuracy curve over the epochs. The validation curve is above the training curve. The last graph from left shows the train and validation curve of the F1 score and the ROC-AUC curve during validation. Again, the training curve lies under the validation curve. The green ROC-AUC curve rises a bit until the third epoch and then flattens at around 0.99

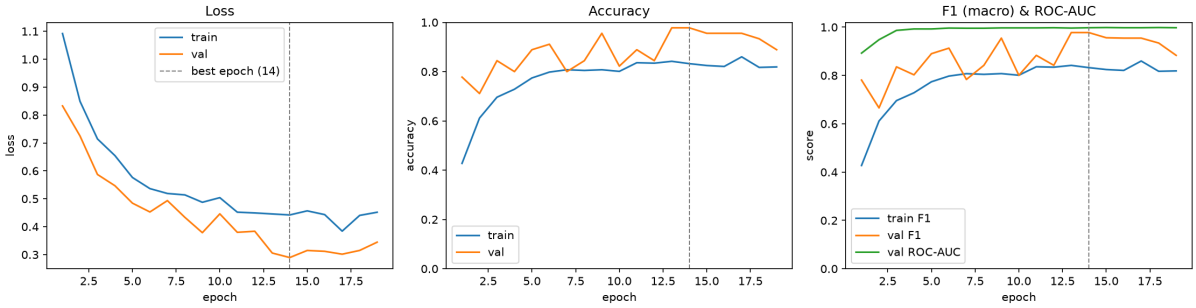


Figure 25: Training Curves

5.2 Choice of Weight Decay

Table 5: Validation loss over three seeds and weight decays

Weight decay	Seed 0	Seed 1	Seed 2	Mean $\pm$ SD
0	0.2895	0.2247	0.2026	0.2389 ± 0.045
10^{-4}	0.2897	0.2251	0.2030	0.2393 ± 0.045
10^{-3}	0.2912	0.2280	0.2073	0.2422 ± 0.044

Table 6: Validation accuracy over three seeds and weight decays

Weight decay	Seed 0	Seed 1	Seed 2	Mean $\pm$ SD
0	0.9777	0.9555	0.9777	0.9703 ± 0.013
10^{-4}	0.9777	0.9555	0.9777	0.9703 ± 0.013
10^{-3}	0.9777	0.9555	0.9777	0.9703 ± 0.013

Nine ($3\ values \times 3\ seeds$) training runs were performed for the weight decay comparison, with weight decay of $\in \{0, 10^{-4}, 10^{-3}\}$. Choice criterion was the mean validation loss over the three seeds. Visualized in table 5.

The three mean validation loss values differ by around 0.0033. As seen in table 6 validation accuracy is identical for all three at 0.970 ± 0.013. However, the variation across the seeds is much larger than the variation across the weight decay values. The seeds having a spread of 0.086. Calculated by subtracting the lowest mean of a seed over all weight decays from the highest mean of a seed over all weight decays. The impact across a seed is 26 times higher than the impact across the weight decay.

5.3 Performance on Test Set

The model was once evaluated with the test set and then the results were frozen. All metrics that were implemented are listed in table 7 once over all three seeds and the mean across all three seeds with their respective standard deviation (SD).

Table 7: Performance on the test set

Metric	Seed 0	Seed 1	Seed 2	Mean $\pm$ SD
Accuracy	0.889	0.911	0.956	0.919 ± 0.034
Precision (macro)	0.904	0.921	0.957	0.927 ± 0.027
Recall (macro)	0.889	0.911	0.956	0.919 ± 0.034
F1 (macro)	0.889	0.911	0.956	0.919 ± 0.034
ROC-AUC (macro, OvR)	0.987	0.984	0.981	0.984 ± 0.003
PR-AUC (macro)	0.975	0.969	0.971	0.972 ± 0.003

Table 8 lists the per-class performance values in precision, recall and ROC-AUC. Each value is the mean over all three seeds with their respective SD.

Table 8: Per-class performance in precision, recall and ROC-AUC

Class	Precision	Recall	ROC-AUC
reception	0.853 ± 0.076	1.000 ± 0.000	1.000 ± 0.000
set	1.000 ± 0.000	0.889 ± 0.038	0.973 ± 0.010
spike	0.928 ± 0.005	0.867 ± 0.067	0.979 ± 0.001
macro average	0.927 ± 0.027	0.919 ± 0.034	0.984 ± 0.003

5.4 Per-Class Performance

The matrix tables in figure 26 visualize which class got confused with which in the test set over all seeds. On the left side (y-axis) comparing true labels against, on the bottom (x-axis), against predicted label.

Over all seeds can be seen that no other classes got misclassified as a set. And reception always got classified correctly.

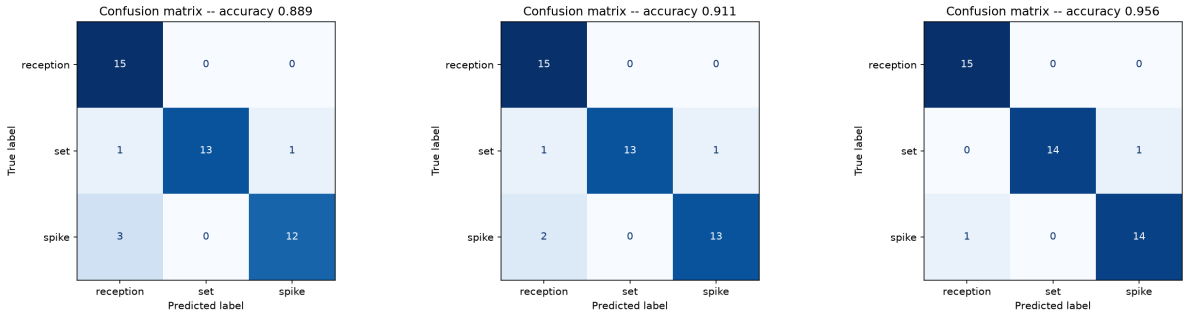


Figure 26: Matrix over all seeds

6 Discussion with Outlook, Reflection

6.1 Robustness of the Data

6.1.1 Trimming and Sampling

The fallback of the implementation of the spike and reception clips was discovered belatedly, after the test runs, and happened due to a configuration mismatch. The window span was too narrow to sample 16 frames. Therefore, the implementation fell back to the initial length of the clip with two seconds (code shown in figure 16).
Table 9 shows how many frames would have been 'available' for sampling.

Table 9: Trim window, frames fall back

Class	Window	Frames at 30̇fps	Frames at 50̇fps	Result
Reception	0.17s	5	8	fallback in both
Set	0.85s	26	42	window used in both
Spike	0.10s	2	5	fallback in both

6.1.2 Data Source Distribution

The distribution of the source of the data was imbalanced. Namely, not the same amount of clips from each class was taken. Presented in table 10.

Table 10: Distribution of source clips

Match	Reception	Set	Spike	Total	fps
1072	20	19	21	60	30 and 50
1074	0	13	21	34	50
1077	22	12	10	44	30
1087	13	27	18	58	30
3812	23	18	15	56	30
3815	22	11	15	48	30
Total	100	100	100	300	

None of the 53 clips recorded at 50 fps is a reception. Frame rate is therefore not independent of the class, and the model could in principle use it as a shortcut feature instead of the action itself.

If I were to do this project again, I would make sure to take the same amount of clips for every action in each game for a more even distribution. I would also check the videos fps to have a normalized frame rate over all video clips.

6.2 Training Performance, Overfitting

The training turned out as anticipated: the loss fell smoothly, without instability and the early stopping was triggered. Therefore, the pipeline worked.

As shown in Figure 25, the validation ROC-AUC rises from 0.89 to about 0.99 within three epochs and then levels off. This indicates that the features produced by the frozen R3D-18 managed to separate reception, set and spike almost completely: the information needed to discern the three actions is present in the feature space, and the HC only has to find a linear boundary through it. Therefore, the remaining epochs do not further improve the clips' ranking. They only refine where the boundary is placed.

The validation curves fluctuate from epoch to epoch because the validation set contains only 45 clips, so a single clip corresponds to 2.2 percentage points. Individual epochs should therefore not be interpreted; only the overall trend and the final level are meaningful.

Figure 25 also shows that validation loss reached its minimum and then rose, while training loss stayed flat. This indicated that the model started overfitting and early stopping got triggered.

The stopping point was partly due to luck. Noisy epochs could have ended training early. The three seeds stopped at epochs 19, 34, 46 with almost identical curves. The validation numbers were optimistic because validation was used for early stopping and the selection of the configuration. The accuracy drop of 0.970 ± 0.013 (table 6) in the validation set to 0.919 ± 0.034 (table 7) in the test set make this optimism visible.
In future, I would increase patience to lessen the possibility of noisy epochs, which could stop training early. Especially on a model with a small validation set.

6.3 Augmentation

As mentioned in chapter (Backbone, Feature Extraction and Augmentation) the augmentation was fixed. An advantage of fixed augmentation is that it is easy to implement. Furthermore, the computational cost was smaller by saving it as vector file, unlike non-fixed augmentation, which would have been saved on disk, instead cached as feature vectors. However, the model was more prone to overfitting. Automated (non-fixed) augmentation is actually the standard for video-augmentation. [45]

An immediate fix to this problem would have been to cache more variants, as the design did not limit the numbers of augmented copies possible. The augmentation would still have been fixed, but fresh-per-epoch sampling would have become negligible.

An option to automate the augmentation would have been to store smaller clips. The current preprocessed clips have a storage size of $44MB$, because 16 frames were at $720p$ stored. Storing the clips at a lower progressive scan would have made automatic augmentation possible, without running the risk of storage over usage. Besides, the time for a training run still would have come at

a low GPU or CPU cost.

6.4 Testing results

6.4.1 Accuracy, Confusion Matrix, F1

Figure 26 visualizes the error pattern of the classification. It is observable in all figures that no clip was ever misclassified as a set, and every reception was classified correctly. The errors occur between set and the two other classes and, most frequently, from spike to reception. An explanation for this error pattern is that reception and spike both had a full clip length, whereas set was the only class that was trimmed. However, it is necessary to point out that the test set was, in total, run on only 45 clips, which is not enough to identify a trend.

A high accuracy and F1 score was achieved, as shown in table 7, over all three seeds. This lead to a suspicion. The suspicion was that the group stratified split lead to too optimistic results as all six recording were distributed over all splits. This effect is called domain shift and can lead to too optimistic evaluation results.

6.4.2 ROC-AUC, Precision, Recall

Table 8 shows that reception has a perfect recall of 1.000 but the lowest precision, 0.853, while set conversely has a perfect precision of 1.000 and a recall of 0.889. This means that the model never misses a reception but over-assigns the label. Neither accuracy nor macro-F1 reveals this.
The same table shows that reception reaches a ROC-AUC of 1.000 in all three seeds. This means that every true reception receives a higher reception probability than every non-reception clip, so the ranking is flawless. Combined with the weakest precision of the three classes, this shows that the mistakes come from the decision rule and not from the feature space: `argmax` compares the three probabilities per clip rather than ranking within a class.

6.5 Generalisation to Unseen Recording

6.5.1 Problem to Idea

The main accuracy result of 0.919 (table 7) came from a group_id split. The same game was distributed over all splits. That means that the model had already seen the gym, players and angles before in training. This does not imply that the venue says anything about which class the right label of the clip is, as all three actions are being played. This effect is known as domain shift.

The feature vector does not only encode the pure motion, it also encodes the space around said motion, meaning how the position appears on the video (camera angle/distance, lighting, coloring of jersey/court and background). Thus, the backbone constructs a feature space. Allowing the HC to place a decision boundary around an action inside it. Hence, the model would classify during the test by construction (feature space and feature of action) and not only by recognizing an action. This

trait can reduce the performance of the model on unseen matches. Therefore, cross-validation was used as an additional analysis, to see how well the model would perform in unseen data. [46]

6.5.2 Method

Five rounds of cross-validation on the model were performed to evaluate how well the model would perform in previously unseen recordings. One round of cross-validation is called a fold. In a fold one slice of the data is withheld and the model is then trained on the rest. The data slice is subsequently used to evaluate the model. The slice of data is in this case one match recording. Recording 1074 was never held-out, as it does not have any clips of reception. During a fold all 1140 feature rows were pooled from all original split files and random split was dissolved. The match id (recording) mattered now for slicing. The data was sliced by the source clips. The test set was made from the withheld recording, where all clips were used to evaluate. Hence, the uneven distribution over all folds and actions (refer to table in chapter xy.). E.g. fold 1077 with 44 clips (22 reception, 12 set, 10 spike). The remaining five match recordings were used to train the model. 15% of the source clips per class were drawn as a validation set for early stopping. The other 85% of the training source clips were, with their augmented copies, used as a train set.

For each fold and seed (5×3), the head classifier was trained from scratch and then evaluated on the held-out slice . 15 training runs with 15 evaluations were made. The augmented clips were only used in the training set, as they would have falsified the test and validation numbers.

Feature extraction was not re-done, as a clips 512-dimensional vector was the same, no matter which fold it landed in. Explaining why 15 runs only took a few minutes.

6.5.3 Results

The graph in figure 27 visualizes the data of table 11. The accuracy drop from random split to the unseen recordings is about 6.5 percentage points (see table 7 and 11).

Table 11: Accuracy per fold with number of clips

Match	Reception	Set	Spike	Total	Accuracy (mean$\pm$SD over 3 seeds)
1072	20	19	21	60	0.978 ± 0.010
1077	22	12	10	44	0.758 ± 0.069
1087	13	27	18	58	0.782 ± 0.026
3812	23	18	15	56	0.899 ± 0.027
3815	22	11	15	48	0.854 ± 0.055
Across all Folds	100	87	79	266	0.854 ± 0.090

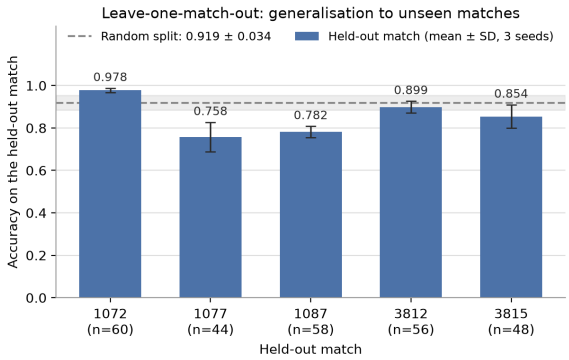


Figure 27: Folds

6.5.4 Discussion

A notable drop in accuracy between random split and unseen recording could be determined. Cross-validation made it somewhat possible to quantify the effect of domain shift. Furthermore, cross-validation showcased that the limiting factor was neither the model, nor the hyperparameters nor the seeds, it was the six recordings. The variation between recordings far exceeds, with a range in accuracy from 0.758 to 0.978, other sources of variation, like the three seed with a distribution of only 0.01 to 0.07.

The 15 runs provide 266 distinct clips evaluated under three seeds, 798 predictions in total — six times more clips than the 45 of the test set, creating more trust in error analysis. Moreover, a pattern could now be interpreted and not only be based on chance findings.
However, it is necessary to take note that a domain shift could have occurred, as only three matches were taken and 6 recordings exist, each team having its own recording of the match. The court, net and background could have already been learned for the test set. Therefore the probability of 0.854 ± 0.090 still being optimistic.

Therefore, for a future project, I would take a completely different match to test the model.

6.6 Reflection

6.6.1 Process

The process can be divided into three stages. The first stage was spent doing extensive research as prior knowledge in the field ML and DL was little. A considerable amount of time had to be dedicated to research and learning. All methodical decisions were made during the first stage. Such as deciding to rely on a frozen backbone, instead of training my own model, the video-augmentation and the structure of the HC. Once the base of the methodology had been determined, the development of product could be started.

In the second stage all code scripts were developed and tweaked with the help of Claude. This stage

was a mix of writing the scripts and research, as constant adjustments to e.g. evaluation metrics or the regularisation and tuning of the HC had to be made.

The dataset was preprocessed, the model trained and evaluated with the guidance of Claude. During the evaluation it was discovered that the evaluation numbers were too optimistic due to the $group_id$ split. Hence, cross-validation was used to re-train and re-check the models performance, which was prior not part of the methodology. With the support of Claude the folds were performed smoothly and could be evaluated. The resulting metric numbers could then be interpreted and compared to $group_split$.

The last stage was the analysis of the metrics and writing the paper. During the writing of the article and cross-verification with Claude, it was discovered that the clips had different fps and the configuration error from the cutting of the spike and the reception.

6.6.2 Custom Model vs. Pretrained Model

A lot of research time was spent on the initial idea of training a model from scratch. Although having been foreseen in the contract, this idea had to be abandoned. A small dataset with 300 video clips would not have been enough, even after augmentation. Training the model and tuning millions of parameters manually would have been outside of the scope of this current project. It was therefore decided that a pretrained model would be used as a feature extractor and only a small classifier head would be added to keep the scope smaller and feasible. Research could be continued as the already gained knowledge had to be applied to the backbone and HC anyway.

6.6.3 Reflecting on the Usage of AI

Overall, it was not planned to rely so heavily on an AI like Claude. Initially, I viewed it merely as a tool to easily offload all my workload and effort. However, this soon proved to be a serious error in reasoning. Rather, AI has to be regarded as a guide, an aide and advisor, playing an indispensable role for developers.

Although scripts were written by Claude, all decisions and executions were made by me. Knowledge and understanding of the scripts was still needed to modify them to my needs. The sheer amount of time it would have taken to write the scripts and debug them would not have been given due to the time restriction of the matura project and being a professional athlete in school.
Claude was used during the model training and the writing of the thesis mainly as a cross-verification tool. This proved to be a good choice, as it had found defects or issues, which would not have been found by me, due to my limited experience and knowledge in this field.

6.6.4 Troubles and Issues

Below a list of all major issues.

Trim windows

The windows for reception (0.17 s) and spike (0.10 s) are shorter than the 16 frames the pipeline samples, so both silently fell back to the full 2-second clip; only the action set was ever trimmed. Next time: An assertion or a warning when a window is too short, so a fallback cannot happen unnoticed.

Change of approach
Training a 3D CNN from scratch was not feasible with 300 clips and the available hardware, so a frozen pretrained backbone was used instead. Next time: Check the data volume against the method before committing to it.

One recording unusable as a fold
Recording 1074 contains no reception clips, so five folds were possible instead of six. Next time: Check class coverage per recording while selecting the matches, not after.

Mixed frame rates
247 clips at 30 fps and 53 at 50 fps, with no reception clips among the 50 fps ones, leaving a problem that could not be solved after the export of the clips. Next time: Fix one frame rate at export.

Validation set too small
45 clips mean a single clip is worth 2.2 percentage points, so individual epochs cannot be interpreted. Next time: More data, or repeated splits instead of one.

Group stratified split:
Six recordings were split across all splits. Making the values of the evaluation accuracy too optimistic. Next time: Split by recording.

The most serious issue was the silent fallback in the trim windows, since the pipeline processed the data differently from what had been specified without reporting it. An error that produces a warning is found immediately; one that quietly changes the input is only found if it is being looked for.

6.6.5 Technical Skills Learned

Various ML and DL concepts were learned and applied in the course of this project. The most essential tool was PyTorch, which was needed to build the head classifier with `nn.Sequential`, to attach an optimizer and a loss function, and to switch the model between training and evaluation mode. Beyond the library itself, concepts such as transfer learning, regularisation and the implementation of evaluation metrics were applied: precision, recall and F1 were not taken from an existing library but computed directly from the confusion matrix.

Just as important were the methodological skills. The data set was divided with a grouped and stratified split in order to avoid data leakage, every configuration was trained with three fixed random seeds so that results could be reported as a mean rather than as a single run, and a cross-validation was designed to answer a specific question, namely how well the model transfers to a recording it

has never seen.

LaTeX is also among the new skills that were learned. It was used to give the document a professional and consistent typesetting, and it handles numbering, cross-references to figures and tables, equations and the bibliography automatically. Initially, the structure of a LaTeX document proved to be challenging to understand and required effort to learn. However, LaTeX is a widely used tool and an important skill to have, which is why I intend to use it for future papers as well.

6.6.6 Time Management, Workload

From the starting point onward, it was clear that it would be a bigger workload. Therefore, good time management was required to handle sports, school and this project. Past experience had shown that it was rather inefficient for me to set a major deadline on a specific week, since a competitive athlete's weekly schedule can vary greatly. Therefore, specific time periods were planned to deeply immerse myself into the project and getting major work done, like school holidays or days without practice. This proved more practical than constantly working every week. It also allowed me to take rest from working on the project during busier periods.

By allowing myself to take several weeks off ambition, motivation and self-discipline could be better preserved. The experience of this project has taught me how to realistically plan projects of such large scale over a long time period.

Furthermore, there was a huge amount of material to study and research, due to little prior knowledge. Through this I learned how to handle large amounts of new information and to retain them.

7 Conclusion

The initial goal of an automated recognition and classification between three volleyball movements: set, spike, reception was met. However, the initial idea of a 3D CNN pipeline made from scratch had to be altered to a pretrained R3D-18 feature exctractor with a custom HC.
The model could successfully discern between the classes. Yet, this estimate was suspected to be too optimistic, since the random, group-wise split placed clips from the same recordings in every split. Hence, a second evaluation was carried out, withholding one entire recording per fold, in order to observe how the model performs with recordings it had never seen before. As suspected, the accuracy of the model after cross-validation dropped in comparison to the stratified split.
Nevertheless, a satisfactory result was reached, while leaving various options for improving this model, enabling continuing research on this project.

Glossary

Term	Explanation
artificial intelligence	The field concerned with building systems that perform tasks normally requiring human intelligence; machine learning and deep learning are nested subfields of it [9].
attacking on the net	Action of a player attacking a ball in the front row [5].
Balltime	A commercial AI volleyball platform that automatically generates statistics and video analysis from match footage [12].
bump	Action of playing a volleyball by underhand passing [4].
cached	Temporarely stored in the fast, local memory of the computer [47]
computer vision	Allowes a machine to analyse visual data [48]
convolutional layer	A layer that slides learned filters across its input and computes dot products, producing feature maps that respond to local patterns [27].
convolutional neural network	A neural network built from convolutional layers that detects local patterns in grid-like data such as images, learning increasingly complex features layer by layer [24].
data leakage	Occurs when information unavailable at prediction time is present during training, producing an over-optimistic estimate of performance [37].
domain shift	The data a model is applied to differs systematically from the data it was trained on, so performance drops [46].
dropout	A regularisation technique that randomly deactivates a fraction of units during training, so the network cannot rely on single features [42].
DVW	File format to import the scouted information from a game in form of python code [49]
feature engineering	Selecting and transforming raw data into variables a model can use effectively [10].
feature extractor	A pretrained network used only to convert inputs into feature vectors, with its weights left unchanged [38].
feature map	The output of applying one filter across the whole input, showing where that filter's pattern occurs [27].
feature space	The multi-dimensional space spanned by the extracted features, in which every input is represented as one point [23].
FIVB	The International Volleyball Federation, which issues the official rules of the game [3].
frame sampling	Selecting a fixed number of frames from a video clip so it can be fed to a network that expects a fixed input length [32].

Term	Explanation
fully connected layer	A layer in which every neuron is connected to all outputs of the previous layer; usually placed at the end of a CNN to map features to class scores [29].
hidden layer	Any layer between the input and the output layer; its values are not observed directly [14].
Hudl	A sports technology company whose Volleymetrics product provides professional volleyball video analysis and statistics [13].
hyperparameter tuning	Searching for the values that are set before training, such as learning rate or dropout rate, that give the best validation performance [23].
input layer	The first layer of a network, which receives the raw input values [14].
kernel	The small learnable filter of a convolutional layer that is slid across the input [27].
learning rate	The hyperparameter controlling how large a step the optimiser takes along the gradient at each update [41].
libero	Specialised player position in the back row only, for reception and defence [50].
linear layer	A layer applying a weighted sum plus a bias to its inputs; the same as a fully connected layer [29].
middle blocker	Specialised player position playing in the front row middle position as well as serving [50].
Open Volley Scout	Open source system with R and Python to read and analyse data DVW-files [49]
opposite	Specialised player position playing on the right side of the net as well as the back row [50].
output layer	The final layer, producing the prediction — one value per class in classification [14].
outside hitter	Specialised player position playing on the left side of the net as well as the back row [50].
passing	The action of playing a ball by underhand passing, mainly in defence and reception [4].
performance degradation	The gradual decline of the performance of a model after deployment [51]
pooling layer	A layer that reduces the spatial size of a feature map by summarising local regions, for example by taking the maximum [27].
rally	A ball played over the net by both teams, starting with the serve until it drops to the ground [3].
reception	The action of receiving a serve during a rally [3].

Term	**Explanation**
ResNet	A CNN architecture using skip connections that let the signal bypass layers, which makes very deep networks trainable [30].
set	The action of playing the (usually second) ball, mostly by overhead pass [4].
setter	Specialised player position setting balls to the attacking players in the front and back rows [50].
setting	Playing a ball by overhead passing, mostly from setter to attacker [4].
shortcut feature	An unintentional easier pattern in the training data which allows the model to use in training instead of the actual pattern it should learn [52].
side-out	Scoring a point and gaining the right to serve the next ball [3].
spike	The action of hitting the ball downwards with a jump [5].
Swiss Volley	Swiss volleyball federation
vanishing gradient	Gradients become progressively smaller as they propagate back through many layers, so the early layers barely learn [30].
scouting	Act of recognizing and evaluation volleyball actions

List of Abbreviations

Abbreviation	Meaning
AI	artificial intelligence
BGR	blue, green, red
CNN	convolutional neural network
DL	deep learning
DVW	DataVolley scouting file format
FC layer	fully connected layer
FIVB	International Volleyball Federation
fps	frames per second
HC	head classifier
ML	machine learning
NN	neural network
R3D-18	18-layer 3D residual network
ReLU	rectified linear unit
ResNet	residual network
RGB	red, green, blue
SD	standard deviation
VM	VolleyMetrics
VS	VolleyStation

Sources

References

1. ZHOU, Guobao; SHI, Xinlong; WANG, Yidong; XU, Kaixin; SHI, Weiquan; LI, Qinglei; PARK, Jaeyoung. Recent advances in the application of artificial intelligence and wearable devices in volleyball. *Frontiers in Sports and Active Living*. 2026, vol. Volume 8 - 2026. ISSN 2624-9367. Available from DOI: 10.3389/fspor.2026.1855108.
2. LIANG, Chen; LIANG, Zhijun. The Application of Deep Convolution Neural Network in Volleyball Video Behavior Recognition. *IEEE Access*. 2022, vol. 10, pp. 125908–125919. Available from DOI: 10.1109/ACCESS.2022.3221530.
3. FIVB. *OFFICIAL VOLLEYBALL RULES 2025-2028* [online]. FIVB, 2025 [visited on 2026-09-27]. Available from: https://www.fivb.com/wp-content/uploads/2025/01/FIVB-Volleyball_Rules2025_2028-EN-v05.pdf.
4. VOLLEYBALLWORLD.COM. *Learn the Vital Techniques of Volleyball* [online]. 2025. [visited on 2026-09-27]. Available from: https://en.volleyballworld.com/blogs/learn-the-vital-techniques-of-volleyball.
5. MATTA, Gylton Da. *Why Is Attacking Important in Volleyball?* [online]. 2026. [visited on 2026-09-27]. Available from: https://sportsedtv.com/blog/why-is-attacking-important-in-volleyball.
6. OTL ORIGINS. *Mastering Volleyball Blocks: Usage Techniques — otlvolleyball* [online]. 2026. [visited on 2026-09-27]. Available from: https://otlvolleyball.com/otl-origins/mastering-volleyball-blocks-usage-techniques/.
7. VOLLEYBALL SCOUT. *How to Scout a Volleyball Match* [online]. [visited on 2026-09-27]. Available from: https://volleyball-scout.com/en/articles/how-to-create-a-report.
8. THE ART OF COACHING VOLLEYBALL. *Scouting the competition* [online]. 2020. [visited on 2026-09-27]. Available from: https://www.theartofcoachingvolleyball.com/scouting-competition/.
9. CHOLLET, Francois. *Chapter 1. What is deep learning? - Deep Learning with Python* [online]. 2017-11. [visited on 2026-09-28]. Available from: https://livebook.manning.com/book/deep-learning-with-python/chapter-1#6.
10. GEEKSFORGEEKS. *Feature Engineering* [online]. [visited on 2026-09-28]. Available from: https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/. Section: Machine Learning.
11. DATAIKU. *A deeper understanding of deep learning* [online]. 2025. [visited on 2026-09-28]. Available from: https://www.dataiku.com/blog/a-deeper-understanding-of-deep-learning?utm_campaign=GLO%20CONTENT%20Golden%20Pages%20FY26&utm_source=emea-adwords&utm_medium=paid-search&gad_source=1&gad_campaignid=

22259713347&gbraid=0AAAAAC1WRxmRZWEiAEEvM1bKxYkxJPZrC&gclid=CjwKCAjw_-3GBhAYEiwAjh9fU
zjq2KVzzatENTT1ndBfw2WxoCYtQQAvD_BwE.

12. BALLTIME. *Balltime from Hudl is the leading AI Volleyball Solution • Hudl* [Hudl] [online]. [visited on 2026-09-29]. Available from: https://www.hudl.com/en_gb/products/balltime.
13. HUDL. *Professional volleyball analytics — Hudl Volleymetrics* [Hudl] [online]. [visited on 2026-09-28]. Available from: https://www.hudl.com/en_gb/products/volleymetrics.
14. GEEKSFORGEEKS. *Introduction To Neural Networks* [GeeksforGeeks] [online]. [visited on 2026-09-28]. Available from: https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/. Section: Deep Learning.
15. GEEKSFORGEEKS. *Forward Propagation in Neural Networks* [GeeksforGeeks] [online]. 2026-07-07. [visited on 2026-09-29]. Available from: https://www.geeksforgeeks.org/deep-learning/forward-propagation-in-neural-networks/. Section: Deep Learning.
16. CAMP, Analytics. *Weights and Biases in Neural Networks Explained in JUST 5 Minutes!* [Online video]. YouTube [visited on 2026-09-29]. Available from: https://www.youtube.com/watch?v=nEt5_8V_wpY.
17. GEEKSFORGEEKS. *Weights and Bias in Neural Networks* [GeeksforGeeks] [online]. 2026-05-13. [visited on 2026-09-29]. Available from: https://www.geeksforgeeks.org/deep-learning/the-role-of-weights-and-bias-in-neural-networks/. Section: Geeks Premier League.
18. GEEKSFORGEEKS. *Activation functions in Neural Networks* [GeeksforGeeks] [online]. 2026-05-12. [visited on 2026-09-29]. Available from: https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/. Section: Misc.
19. LANG, Christian. *Why Neural Networks Have Activation Functions* [Towards Data Science] [online]. 2021-02-26. [visited on 2026-09-29]. Available from: https://towardsdatascience.com/why-neural-networks-have-activation-functions-9732e5405d4e/.
20. STANFORD HAI. *What is a Loss Function? — Stanford HAI* [online]. [visited on 2026-10-01]. Available from: https://hai.stanford.edu/ai-definitions/what-is-a-loss-function.
21. TECHNOLOGY, IBM. *What is Back Propagation* [Online video]. YouTube. Available also from: https://www.youtube.com/watch?v=S5AGN9XfPK4&t=186s.
22. 3BLUE1BROWN. *Backpropagation, intuitiv — DL3* [Online video]. YouTube. Available also from: https://www.youtube.com/watch?v=Ilg3gGewQ5U.
23. GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.
24. IBM. *What are Convolutional Neural Networks? — IBM* [online]. 2021-10-06. [visited on 2026-10-03]. Available from: https://www.ibm.com/think/topics/convolutional-neural-networks.

25. OREOLUWA, Bolarinwa. *Understanding 3d Convolution and Some of it's Application to AI systems.* [Rectlabs Inc] [online]. 2024-11-23. [visited on 2026-10-03]. Available from: https://medium.com/rectlabs/understanding-3d-convolution-and-some-of-its-application-to-ai-systems-df7be420ecb5.

26. TRAN, Du; WANG, Heng; TORRESANI, Lorenzo; RAY, Jamie; LECUN, Yann; PALURI, Manohar. *A Closer Look at Spatiotemporal Convolutions for Action Recognition* [online]. arXiv, 2018 [visited on 2026-10-03]. No. arXiv:1711.11248. Available from DOI: 10.48550/arXiv.1711.11248.

27. JAY WANG AND ROBERT TURKO AND OMAR SHAIKH AND HAEKYU PARK AND NILAKSH DAS AND FRED HOHMAN, MINSUK KAHNG, POLO CHAU. *CNN Explainer* [CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization] [online]. 2020-08-28. [visited on 2026-10-03]. Available from: https://poloclub.github.io/cnn-explainer/.

28. MASSOBRIO, Anthony. *3D CNN Guide for Engineers — Neural Concept* [Neural Concept] [online]. 2026-10-01. [visited on 2026-10-03]. Available from: https://www.neuralconcept.com/post/3d-convolutional-neural-network-a-guide-for-engineers.

29. GEEKSFORGEEKS. *What is Fully Connected Layer in Deep Learning* [GeeksforGeeks] [online]. 2026-05-13. [visited on 2026-10-03]. Available from: https://www.geeksforgeeks.org/deep-learning/what-is-fully-connected-layer-in-deep-learning/. Section: Blogathon.

30. GMI CLOUD. *Understanding ResNet in Deep Learning Models* [GMI Cloud] [online]. [visited on 2026-10-03]. Available from: https://www.gmicloud.ai/en/glossary/resnet.

31. CODEEMPORIUM. *ResNet - explained!* [Online video]. YouTube. Available also from: https://www.youtube.com/watch?v=gyhCfjixLV0&t=243s.

32. PETHERBRIDGE-CONROY, Jamie. *Preparing Video Data for Deep Learning: Introducing Vid Prepper — Towards Data Science* [Towards Data Science] [online]. 2025-09-29. [visited on 2026-10-03]. Available from: https://towardsdatascience.com/introducing-vid-prepper/.

33. COMET, Team. *Why is Model Evaluation Important in Machine Learning?* [Comet] [online]. 2022-11-02. [visited on 2026-10-03]. Available from: https://www.comet.com/site/blog/why-is-model-evaluation-important-in-machine-learning/.

34. GOOGLE FOR DEVELOPERS. *Classification: Accuracy, recall, precision, and related metrics — Machine Learning* [Google for Developers] [online]. [visited on 2026-10-15]. Available from: https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall.

35. GEEKSFORGEEKS. *Machine Learning Model Evaluation* [GeeksforGeeks] [online]. 2026-05-13. [visited on 2026-10-03]. Available from: https://www.geeksforgeeks.org/machine-learning/machine-learning-model-evaluation/. Section: Technical Scripter.

36. PYTORCH. *r3d_18 — Torchvision main documentation* [online]. [visited on 2026-10-04]. Available from: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.video.r3d_18.html.

37. IBM. *What is Data Leakage in Machine Learning? — IBM* [online]. 2024-09-30. [visited on 2026-10-15]. Available from: https://www.ibm.com/think/topics/data-leakage-machine-learning.

38. WUTTKE, Laurenz. *Transfer Learning: Anwendungsfälle und Beispiele* [Datasolut GmbH] [online]. 2023-05-24. [visited on 2026-10-15]. Available from: https://datasolut.com/was-ist-transfer-learning/.

39. IBM. *What is Overfitting? — IBM* [online]. 2021-10-15. [visited on 2026-10-04]. Available from: https://www.ibm.com/think/topics/overfitting.

40. PYKES, Kurtis. *Cross-Entropy Loss Function in Machine Learning: Enhancing Model Accuracy* [online]. 2026-02-27. [visited on 2026-10-04]. Available from: https://www.datacamp.com/tutorial/the-cross-entropy-loss-function-in-machine-learning.

41. KINGMA, Diederik P.; BA, Jimmy. *Adam: A Method for Stochastic Optimization* [online]. arXiv, 2017 [visited on 2026-10-04]. No. arXiv:1412.6980. Available from DOI: 10.48550/arXiv.1412.6980.

42. GEEKSFORGEEKS. *Dropout Regularization in Deep Learning* [GeeksforGeeks] [online]. 2025-12-19. [visited on 2026-10-04]. Available from: https://www.geeksforgeeks.org/deep-learning/dropout-regularization-in-deep-learning/. Section: Deep Learning.

43. GEEKSFORGEEKS. *Regularization by Early Stopping* [GeeksforGeeks] [online]. 2026-07-15. [visited on 2026-10-04]. Available from: https://www.geeksforgeeks.org/machine-learning/regularization-by-early-stopping/. Section: Machine Learning.

44. VAGHASIYA, Nikunj. *Understanding Seeds in AI: The Key to Reproducibility and Creativity* [Medium] [online]. 2024-11-22. [visited on 2026-10-05]. Available from: https://medium.com/@nikunj.vaghasiya2050/understanding-seeds-in-ai-the-key-to-reproducibility-and-creativity-edcfd3bf649c.

45. KAMIL TOKARSKI, KRZYSZTOF ŁECKI. *Why Automatic Augmentation Matters* [NVIDIA Technical Blog] [online]. 2023-05-05. [visited on 2026-10-11]. Available from: https://developer.nvidia.com/blog/why-automatic-augmentation-matters/.

46. TORRALBA, Antonio; EFROS, Alexei A. Unbiased look at dataset bias. In: *CVPR 2011* [online]. Colorado Springs, CO, USA: IEEE, 2011, pp. 1521–1528 [visited on 2026-10-15]. ISBN 978-1-4577-0394-2. Available from DOI: 10.1109/CVPR.2011.5995347.

47. CLOUDFLARE. *What is Caching? — How is a Website Cached?* [online]. [visited on 2026-10-15]. Available from: https://www.cloudflare.com/learning/cdn/what-is-caching/.

48. IBM. *Was ist Computer Vision? — IBM* [online]. 2025-09-17. [visited on 2026-10-15]. Available from: https://www.ibm.com/de-de/think/topics/computer-vision.

49. DATAVOLLEY. *Reading and Analyzing DataVolley Scout Files* [online]. [visited on 2026-10-15]. Available from: `https://datavolley.openvolley.org/`.

50. VOLLEYBALLWORLD.COM. *What are the Positions and Roles in Volleyball?* [online]. 2025. [visited on 2026-09-27]. Available from: `https://en.volleyballworld.com/blogs/what-are-the-positions-and-roles-in-volleyball-x5886`.

51. SCIENTIST/MLE), Sahin Ahmed(Data. *Why does machine learning model performance degrade, and how can we detect and prevent it?* [Medium] [online]. 2024-07-04. [visited on 2026-10-15]. Available from: `https://medium.com/@sahin.samia/why-does-machine-learning-model-performance-degrade-and-how-can-we-detect-and-prevent-it-70f546a54548`.

52. SERPA, Ygor. *Shortcut Learning, The Reason ML Models Often Fail in Practice* [TDS Archive] [online]. 2021-10-11. [visited on 2026-10-15]. Available from: `https://medium.com/data-science/shortcut-learning-how-and-why-models-cheat-1b37575a159`.

Hilfsmittelverzeichnis

@p0.2X@

Tool, software, etc.	**Application**
Claude (Anthropic), `claude-opus-5`	Methodological advice on metrics, tuning and evaluation design; generation of all Python, JSON, Markdown and CSV scripts/files (see GitHub repository); wording of the glossary definitions and the abbreviation expansions, based on the term, abbreviation and source lists provided by the author; LaTeX code for these two lists; analysis of the author's own measurement data; structural suggestions; language revision of sections written by the author
PyTorch / torchvision	Model, training, pretrained R3D-18
OpenCV, NumPy, scikit-learn, Matplotlib	Video decoding, arrays, ROC-AUC / PR-AUC, figures
VolleyMetrics, VolleyStation	Video footage, scouting files, filtering and export of the clips
LaTeX, Git	Typesetting of the thesis, version control

List of Figures

1 Volleyball basic positions . . . 1
2 Setter on Position 6 . . . 2
3 Manual scouting live during a game . . . 4
4 Overview of relation between AI, ML and DL . . . 5
5 Structure of a NN . . . 6
6 Structure of a CNN . . . 7
7 Kernel sliding over 3D data . . . 8
8 Structure of a ResNet model . . . 9
9 First Step . . . 12
10 Second step . . . 12
11 Third step . . . 13
12 Fourth step . . . 13
13 Choice of filter for the first class . . . 14
14 Choice of filter for the second class . . . 14
15 Choice of filter for the third class . . . 15
16 Code for resetting trim if 16 frames are not guaranteed . . . 16
17 Reception . . . 16
18 Set . . . 16
19 Spike . . . 16
20 Pipeline . . . 18
21 Architecture of the R3D-18 . . . 19
22 Code of freezing the backbone and turn into eval-mode . . . 20
23 Structure of head classifier . . . 21
24 CrossEntropyLoss and Adam optimizer . . . 21
25 Training Curves . . . 24
26 Matrix over all seeds . . . 26
27 Folds . . . 31

List of Tables

1 List of games with respective recording 11
2 Aufteilung des Datensatzes nach Split und Aktion. 18
3 Hyperparameters of R3D-18 Backbone 22
4 Seeds, best epoch, stopping epoch. 24
5 Validation loss over three seeds and weight decays 24
6 Validation accuracy over three seeds and weight decays 25
7 Performance on the test set 25
8 Per-class performance in precision, recall and ROC-AUC 26
9 Trim window, frames fall back 27
10 Distribution of source clips 27
11 Accuracy per fold with number of clips 30

Materials